



BSc (Hons) Computer Science

CIS013-3 Research Methodologies and Emerging Technologies

# **Fixora: A Multi-Agent Artificial Intelligence-Based Automated Administrative System for Enterprise Business Operations**

## **Contextual Report**

Student Name: R M I T Rathnayake

Student ID: 2521838

Date of Submission: 15<sup>th</sup> May 2026

## Table of Contents

|                                                                          |     |
|--------------------------------------------------------------------------|-----|
| Table of Contents .....                                                  | i   |
| List of Figures .....                                                    | iii |
| List of Tables.....                                                      | iv  |
| 1. Chapter 1: introduction .....                                         | 1   |
| 1.1. Project Background.....                                             | 1   |
| 1.2. Aim .....                                                           | 2   |
| 1.3. Objectives .....                                                    | 2   |
| 1.4. Description of the Artifact .....                                   | 2   |
| 1.5. Structure of the Report.....                                        | 3   |
| 2. Section 2: Literature Review & Market Research .....                  | 4   |
| 2.1. Literature Review.....                                              | 4   |
| 2.1.1. Introduction.....                                                 | 4   |
| 2.1.2. Multi-Agent Systems (MAS) in Enterprise Applications .....        | 5   |
| 2.1.3. Artificial Intelligence in IT Operations (AIOps) .....            | 5   |
| 2.1.4. Natural Language Interfaces (NLI) for System Administration ..... | 6   |
| 2.1.5. Identifying the Research Gap.....                                 | 7   |
| 2.2. Market Research .....                                               | 8   |
| 2.2.1. Method .....                                                      | 8   |
| 2.2.2. Results.....                                                      | 9   |
| 2.2.3. Conclusion .....                                                  | 10  |
| 3. Section 3: Project Plan .....                                         | 12  |
| 3.2. Description of the Project Plan .....                               | 12  |
| 3.3. Gantt Chart.....                                                    | 13  |
| 3.4. Work Breakdown Structure (WBS) .....                                | 14  |
| 4. Section 4: Planning Your Artifact .....                               | 15  |
| 4.1. Methodology .....                                                   | 15  |
| 4.2. Planning .....                                                      | 15  |
| 4.3. Requirement Gathering and Analysis .....                            | 16  |
| 4.4. Design .....                                                        | 16  |
| 4.5. Implementation .....                                                | 18  |
| 4.6. Risks and Risk Mitigation.....                                      | 20  |

|     |                                                     |    |
|-----|-----------------------------------------------------|----|
| 5   | References.....                                     | 23 |
| 6   | Appendices.....                                     | 24 |
| 6.1 | Appendix A: Questionnaire/Interview Questions ..... | 24 |

## **List of Figures**

Figure 1: Gantt Chart – Fixora

Figure 2: Work Breakdown Structure – Fixora

Figure 3: Survey Data Collection Interface (Google Sheets)

Figure 4: Role Distribution of Respondents

Figure 5: Top Requested AI-Driven Features

Figure 6: Weekly Time Lost due to System Inefficiencies

Figure 7: Perceived Usefulness of AI-Based Error Explanations

Figure 8: SWOT Analysis for Fixora Project

## **List of Tables**

Table 1: Risk Assessment and Mitigation Strategies

## **1. Chapter 1: introduction**

### **1.1. Project Background**

In today's corporate world, digital ecosystems in enterprises have become increasingly intricate and interwoven. Companies tend to employ a diverse array of relational databases such as MySQL and PostgreSQL, interactive web applications built with React or Angular, and a range of backend microservices to handle critical functions such as payments or authentication. This complexity not only allows for unprecedented scalability and redundancy but also presents a new management challenge. The amount of data produced by these systems, including transaction logs, server telemetry and API liveness, is vast. Unfortunately, many companies have struggled to keep up with the pace of change in the management of these systems. The aim of the "Fixora" project is to create an advanced, automated administration system that translates complex system metrics into human-friendly operations. Using cutting-edge artificial intelligence, Fixora will help translate complex technical telemetry into actionable knowledge, making the state of an organisation's digital infrastructure accessible and manageable.

The motivation for this research is the growing "technical debt" and "operational friction" in middle-to-large enterprises. Technical debt can be seen in systems that are hacked together, resulting in brittle systems that are hard to maintain. Additionally, operational friction is caused by the interruption of business processes due to technical issues that take too long to diagnose and fix. Existing monitoring systems are often just streams of raw data. They are complex and require a high degree of technical expertise to interpret and understand, so that when a system issue arises, only a few individuals in an organisation are able to comprehend what is happening. The driving idea behind Fixora is to "democratize" system administration. Through a conversational AI platform, Fixora gives users, from project managers to department heads, who may not have extensive IT experience, the ability to monitor system status in real-time. This relieves the burden on existing IT teams, who are often overwhelmed by basic diagnostic inquiries and troubleshooting.

## **1.2. Aim**

The primary aim of this project is to design, implement and evaluate a Multi-Agent AI system, collectively named Fixora, that automates key routine administrative tasks and offers intelligent fault diagnosis in enterprise systems. The project will address the shortcomings of the current threshold-based alert systems by developing an autonomous reasoning and collaborative fault diagnosis system.

## **1.3. Objectives**

In pursuit of this aim, the project has four key objectives. The first objective is to undertake a survey and primary research of more than 80 industry stakeholders. This will establish the specific challenges in existing administrative tasks, including communication barriers between IT and business operations, the most common types of system faults, and the amount of time spent on manually analysing system log data. The second objective is to design a scalable architecture for a community of AI agents, such as a Monitoring Agent, Diagnostic Agent and Retrieval Agent. This objective is concerned with the "social" nature of AI, where agents can share information with each other to gain a holistic view of system health. The third objective is to design and build a prototype system that uses Large Language Models (LLMs) to convert natural language queries into system commands or database queries. This will require fine-tuning or prompting the models to incorporate particular technical terms while sounding conversational. The fourth objective is to thoroughly test the system, with the key performance indicator being a decrease in "Time to Resolve" (TTR) for typical enterprise issues. The assessment will also examine the correctness of the AI's diagnostic recommendations and the system's ease of use for non-technical users, and ensure that it meets its promise of "democratizing administration".

## **1.4. Description of the Artifact**

Fixora is a Multi-Agent Artificial Intelligence-Based Automated Administrative System designed to bridge the gap between technical system data and human administration of faults. The artifact employs a modular design based on the LangChain framework, orchestrating three specific agents. The Monitoring Agent acts as the

vigilant front line, using a combination of traditional rule-based monitoring and advanced anomaly detection using AI. The Diagnostic Agent is responsible for the heavy-lifting "forensic" work, automatically analysing logs and explaining errors in human-readable language. The Retrieval Agent serves as a natural language interface to the structured data of the business, allowing non-technical users to query databases using plain English. The architecture uses Large Language Models (LLMs) like Grok or GPT-4o, which can explain technical telemetry in "layman terms," enabling non-technical users to interpret and respond to system health information. Initial testing indicates that through automation of the initial fault diagnosis stage and intelligent suggestions via Retrieval-Augmented Generation (RAG), Fixora can dramatically decrease the "Time to Resolve" (TTR) for typical enterprise faults while requiring minimal computational resources.

### **1.5. Structure of the Report**

This report is structured as follows. Chapter 1 provides an introduction to the project, including background, aim, objectives and artifact description. Section 2 presents a literature review on Multi-Agent Systems, AIOps and Natural Language Interfaces, followed by market research methodology, results and conclusions. Section 3 outlines the project plan including Gantt chart and Work Breakdown Structure. Section 4 details the planning of the artifact, covering methodology, requirement gathering, design, implementation, testing strategy and risk mitigation. References and Appendices conclude the report.



## **2. Section 2: Literature Review & Market Research**

### **2.1. Literature Review**

#### **2.1.1. Introduction**

This literature review explores three themes that are important to the Fixora project: Multi-Agent Systems (MAS) in enterprise applications, Artificial Intelligence for IT Operations (AIOps), and Natural Language Interfaces (NLI) for system administration. The review highlights the existing research gaps and introduces Fixora as a novel research contribution. The application of 2.1.2 Multi-Agent Systems (MAS) in enterprise. Multi-Agent Systems (MAS) are an important break from monolithic, centralised software designs to a distributed intelligence and decision-making system. In a typical monolithic system, all the data is fed into a single control loop, or a single logic processing unit which process all the data and make decisions. However, with increasingly complex enterprise systems, this type of monolithic system can no longer handle a variety of data streams from various sources. A MAS, on the other hand, consists of several independent or semi-independent "agents", computational entities that: acquire information from the environment through sensors or data feeds; use local reasoning to interpret this information; move into the world to perform some action that can lead to a desired outcome, either for the agent individually or for the group collectively. Jennings and Wooldridge (1970) noted the flexibility, modularity and social ability of MAS as the main benefits. This architectural style allows a very effective "division of labor" in an enterprise setting. For instance, one agent can be dedicated to watching the health and performance of the database, and another one can be dedicated to watching API response times and microservice connections. The agents are not independent units but are subject to a "coordinator" or "orchestrator" agent that interacts with the agents, resolves between their objectives, and combines their outcomes to deliver a system-level health report. This distributed nature also provides some resilience in the event part of the system becomes too complex or fails, while the other agents can continue working. With the Fixora project, MAS provides an organizational structure that allows the complexity of modern IT administration, monitoring and managing multiple systems like ERPs, web servers and cloud storage at the same time. This course will explore how AI is being used in IT operations.

### **2.1.2. Multi-Agent Systems (MAS) in Enterprise Applications**

Multi-Agent Systems (MAS) represent a significant departure from monolithic, centralized software designs to a distributed intelligence and decision-making system. A typical monolithic system features a single control loop or a central logic processing unit that processes all the data and makes decisions. But as enterprise systems become more complex, such monolithic systems often cannot cope with diverse and distributed data streams from multiple sources. By contrast, a MAS comprises multiple autonomous or semi-autonomous "agents" - computational entities that are able to perceive the environment via sensors (or data feeds), process the information using local reasoning, and act to achieve certain (individual or collective) goals. As noted in the seminal paper by Jennings and Wooldridge, the key advantage of MAS is its flexibility, modularity and social ability. In an enterprise setting, this style of architecture enables a very effective "division of labor". For example, an enterprise network may have one agent that is dedicated to managing the health and performance of the database, while another agent is responsible for API response times and microservice connections. These agents don't work in isolation; they are overseen by a "coordinator" or "orchestrator" agent which communicates with them, arbitrates between their goals, and aggregates their results into a system-level health report. This distributed nature means that if part of the system becomes too complex or breaks down, the other agents can still operate, offering a degree of resilience not found in monolithic systems. In the case of the Fixora project, MAS offers the organizational framework needed to manage the complexity of modern IT administration - monitoring and managing multiple systems such as ERPs, web server and cloud storage simultaneously.

### **2.1.3. Artificial Intelligence in IT Operations (AIOps)**

The rise of AIOps or Artificial Intelligence for IT Operations (AIOps) represents the use of sophisticated machine learning and big data technologies to analyse the wealth of data generated by today's IT systems. Traditionally, IT monitoring has been based on static thresholds, such as alarming when the CPU load is above 90%. AIOps goes beyond these limitations by leveraging historical data to calculate baselines in order to detect "anomalies" that may not breach a static threshold, but are deviations from the

norm. This is brilliant for pattern matching, such as recognising traffic spikes that occur as part of a seasonal cycle or early signs of deteriorating hardware in a cluster of servers. But according to current academic and industry research, there is a gap here: although AIOps is excellent at detecting that an anomaly has taken place, it needs to be complemented with the "reasoning" stage to find root causes in complex, multi-step troubleshooting scenarios. It can alert us to an increase in error logs but may not be able to explain why this occurred. Enter the Fixora project. Fixora's innovation is to build upon foundational AIOps principles (data ingestion and anomaly detection) and layer on a "linguistic reasoning layer" so that it can move from a statistical system to a diagnostic one. Rather than simply presenting the IT administrator with a dashboard of red graphs, the introduction of AI means the system can not only "see" that there is an error, but also "understand" the problem from the log files and communicate in natural language. This shift from recognition to reasoning is critical for decreasing IT workloads, which tend to be disproportionately focused on manual root-cause analysis.

#### **2.1.4. Natural Language Interfaces (NLI) for System Administration**

The rapid and recent advances in Large Language Models (LLMs) have changed Natural Language Interfaces (NLI) from "toys" to fast and efficient tools for managing systems in enterprises. Until recently, communicating with a server or database required the use of intricate command-line interfaces (CLI) or structured query languages (SQL). This posed a significant barrier to entry for non-technical managers or junior employees, who would need to request senior engineers to provide them with information about the status of a server or to perform basic data queries. With the advent of NLIs, this aspect of the work process is transformed, as users can now interact with their systems in their own natural language. One key element of this interface in the Fixora architecture is Retrieval-Augmented Generation (RAG). RAG enables the AI to access real-time data sources, like current logs or active database tables, in addition to the static data it has been trained on. For example, if a user queries, "Why was the system slow at 2:00 PM?", the NLI doesn't merely make an educated guess; the RAG component retrieves the relevant log data from that time period and passes it to the LLM. The AI model then synthesises and summarises this low-level information to

create a human-friendly response, like: "At 2:00 PM, there was a deadlock in the payment processing database caused by a high volume of concurrent transactions." This feature greatly reduces the skill barrier for stakeholders. It enables anyone to access system information, ensuring that decision-makers aren't "flying blind" when system problems occur, and enables more junior IT staff to answer complex queries without having to constantly check with senior experts.

#### **2.1.5. Identifying the Research Gap**

An extensive survey of existing work highlights a clear lack in the current state of IT management systems. At one end of the spectrum there is a lot of research on high-level monitoring systems that are great at collecting data and visualizing metrics, but lack any interactive or diagnostic functionality. On the other hand, there is an abundance of stand-alone AI chatbots that are able to answer basic technical questions or provide summaries of technical documentation but not integrated to the live, real-world IT infrastructure of a particular customer. There is little research and development into truly "integrated" systems where multiple agents can collaborate to achieve what can be called "active administration". Active administration is not about merely informing the user of a problem but the system being able to proactively investigate, relate problems across various layers of the system (e.g. a web error to a database delay) and then automatically offer a specific solution or even make a database call on behalf of the user while the user is waiting for an answer. Today's systems are generally "passive" - they wait for a user to inquire or an event or threshold to be breached. The gap in research is the integration of MAS, AIOps, and NLI to create a unified framework for an "administrative assistant". Fixora aims to address this gap by investigating how a multi-agent hierarchy can handle the "alerting" to "resolving" process. Through an analysis of the coordination protocols between a Monitoring Agent and a Diagnostic Agent, this research aims to set a new benchmark for enterprise IT asset management, transitioning from manual and specialised work flows to the new paradigm of smart, conversational and autonomous system management.

## **2.2 Market Research**

### **2.2.1 Method**

In order to build a solid empirical basis for the Fixora project, a primary research phase was launched by deploying a detailed questionnaire. The overall objective of the market research was to verify the main hypothesis that the level of technical competence required to administer enterprise systems and the level of technical competence of both the technical staff and the non-technical staff who use the system on a regular basis, differ. Through the collection of empirical data from practising professionals, the aim was to dispel theoretical speculations and to base the design of the multi-agent AI system on practical operational concerns. A large number of people are currently working in a variety of different roles in the enterprise and the survey managed to obtain responses from a sample of 84 (N=84). Two demographical groups were targeted, in order to have a rich and varied perspective across the whole range of the corporate area. The first, which made up 59.5 percent of the survey participants, were “Business and Non-IT Users.” This group consisted of project managers, administrators, financial analysts, departmental heads, who are heavy users of enterprise systems software for their work, but do not have traditional systems engineering or database management training. The aim of the outreach to this group was to quantify users' frustration and lost productivity due to technical outages and the sometimes confusing terminology of standard error messages. Software engineers, system administrators, database architects and DevOps professionals make up the other 40.5 percent of survey respondents, known as "IT Staff and Developers". This survey of these experts aimed to understand the macro level issues with the current monitoring and diagnostic tools, as well as the "cognitive load" that the expert experiences during a crisis and how the manual analysis of logs and root cause determination affects his/her productivity. The split of these experts allowed for a comparative study to be taken, showing the "friction points" as areas of technical problems from different departments.

### 2.2.2 Results

The quantitative analysis, which was drawn from the results of the fill-in survey, shed light on the existing issues associated with enterprise management processes. One of the most notable areas for concern was the lack of ability to identify the root causes. 88.2 percent of the IT professionals who were surveyed said that "Moderately Difficult" to "Very Difficult" is the difficulty level in finding the actual cause of a system failure. This is important because it suggests that IT experts are lacking in visibility to a level that is preventing them from fixing problems in a timely fashion with the available tools. Even experts find it difficult to pinpoint an issue quickly with the existing dashboards and tools, and this is an obvious need for an AI-based diagnostic agent to automate the log and metrics correlation to find out the root cause of a failure with one eye closed. Moreover, the survey revealed a huge gap in business efficiency in relation to IT bottlenecks. 86% of end-users said "the IT staff are too busy" is a significant cause of delays in their operations. The more complicated the system is, the more time IT will spend manually troubleshooting; the more time they spend manually troubleshooting, the less time to support the rest of the business, the less they can support the rest of the business, the more downtime and the more requests for support. This "bottleneck effect" reveals the unsustainability of current human dependent administration approach. When the issue of solutions was raised with the respondents, they had very similar demands on future technology. The top feature that the respondents wanted was "Automatic Error Detection", with 92 percent agreeing with this feature. The second most desired feature was "Suggested Solutions", which had 88 percent approval. The ratings indicate that there is a good market opportunity for a system which can identify a problem for a user, as well as solve it. Considering this is the highest priority requirement for both technical and non-technical people, a multi-agent system such as Fixora, which provides both automatic error detection and solution suggestions, seems to meet an important need for the workforce. The questionnaire has a layout similar to that in Figure 3 (Appendices).

The quantitative data provided the "what" and "how much", the qualitative data provided the "why" and the human factor were provided through the open-ended survey responses. System complexity was a prominent theme in the qualitative comments.

Many survey respondents noted that enterprise software is becoming more complicated to the point where transactions are taking longer and therefore impacting day-to-day productivity. It's not just a technical issue but a business issue. The intricacies and time-consuming nature of a system in fact make the work of the system members more difficult and slower to perform their essential tasks, thereby slowing down the process of the business. The most heart-wrenching answers to the survey were from the non-technical users relating to the "language of errors". Many participants expressed a sense of "cultural separation" from the usual technical answers. A clear and strong preference was expressed for providing feedback to the system in "plain English". Participants explained that a code error, like a stack trace or SQL error code is of no help and makes them feel even more frustrated and despondent. They want a "translator", that can accept information about a failure of a system, and re-express it in a way that allows them to move on to the next step, for example "The database is currently being backed up; please retry your request in five minutes". In this qualitative review of feedback, there is strong evidence that multi-agent and natural language capabilities need to be developed in the Fixora project. People don't need a more responsive system, they need a more communicative system. The answers reveal that Natural Language is not a nice to have, but a must have for future enterprise applications.

### **2.2.3 Conclusion**

The market research has validated the project's need and outlined specific features and experiences the AI agents will be expected to deliver. The quantitative data reflects that there are significant pain points with regards to root cause identification, as 88.2 percent of IT professionals report to be having trouble with this, and that 86 percent of end-users have IT bottleneck delays as a result of this. The results of all groups surveyed indicated high levels of support for "Automatic Error Detection" and "Suggested Solutions" which are indicative of a general enthusiasm for a new generation of automated, conversational diagnostic systems. The qualitative feedback is quite consistent in terms of the need for natural language explanations of technical errors: users expressed a strong preference for "plain English" communication over cryptic error codes. Fixora's focus on simplicity of language and automated reasoning

lends itself well to the needs of professionals who've had enough of technology jargon and cumbersome troubleshooting. This market research has not only validated the need for this project but also outlined specific features and experiences that the AI agents expected to offer.



### **3 Section 3: Project Plan**

#### **3.2 Description of the Project Plan**

The development of the Fixora project is an orderly process with a planned duration of approximately 32 weeks, from February to September 2026. The project consists of five major phases. Phase 1 is the Project Planning and Initiation Phase (Weeks 1-7), which involves topic selection, scope definition, project proposal submission, ethics clearance, and a feasibility study. The 2nd phase is called Requirement Gathering and Analysis phase (Weeks 2–7), which covers a literature review of Multi-Agent Systems, AIOps, and LLMs, a market research survey collecting eighty-four responses, statistical analysis of IT pain points, and contextual report preparation. Phase 3 is the Development phase (Weeks 11–26), structured as three Agile sprints: Sprint 01 covers system architecture and UI planning, Sprint 02 implements the Monitoring and Analysis agents for log ingestion and anomaly detection, and Sprint 03 delivers the LangChain orchestration layer, a natural language data retrieval agent, a flutter or React-based system health dashboard, and prototype finalization. Phase 4 is the Testing and Evaluation phase (Weeks 27–32), which involves unit and integration testing, User Acceptance Testing, and performance benchmarking for latency, load, and diagnostic accuracy. Phase 5, Final Report and Documentation, runs concurrently throughout the project and covers all progress reports, the contextual report, and the final thesis submission. The project adopts an Agile-inspired development methodology with two-week sprints to ensure flexibility and iterative progress.

3.3 Gantt Chart

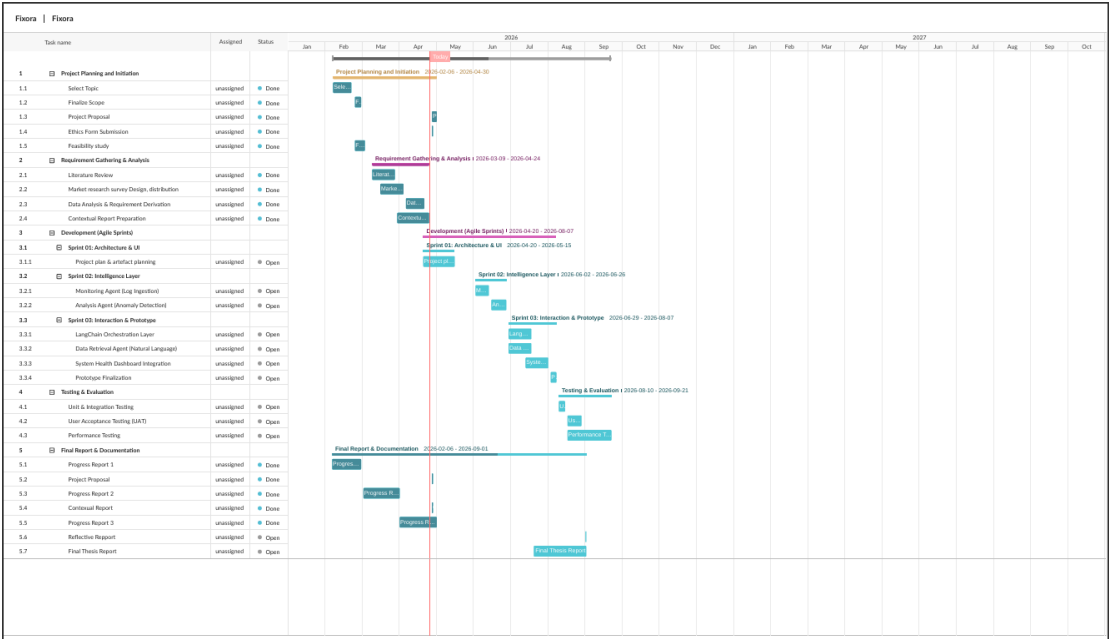
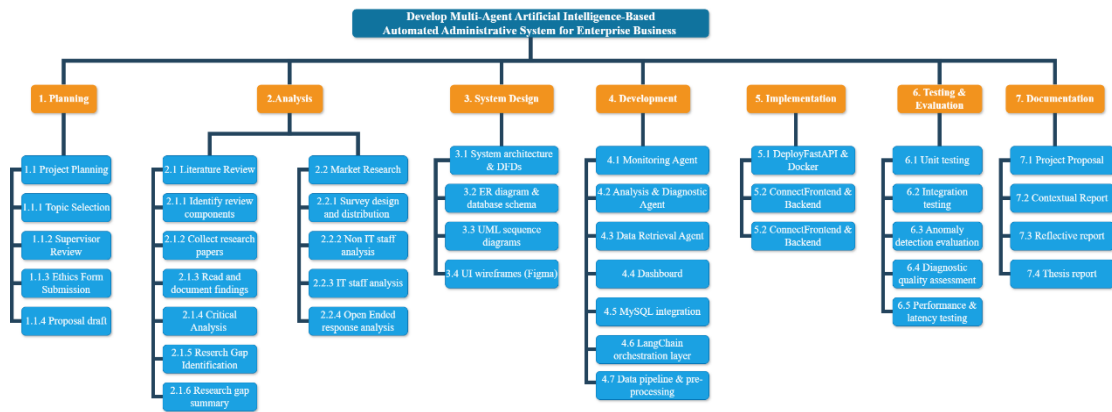


Figure 1: Gantt Chart – Fixora Multi-Agent System Development

### 3.4 Work Breakdown Structure (WBS)



**Figure 2:** Work Breakdown Structure – Fixora Multi-Agent System Development

The Work Breakdown Structure of the Fixora project is divided into five big Work-Packages. The first work package is Planning, which involves topic selection, supervisor review, ethics form submission, and proposal drafting. The second work package is Analysis, which is further divided into Literature Review and Market Research. The literature review covers identifying review components, collecting research papers, documenting findings, critical analysis, research gap identification, and gap summary, while the market research includes survey design and distribution, non-IT staff analysis, IT staff analysis, and open-ended response analysis. The third work package is System Design, it include system architecture and data flow diagrams, ER diagram and database schema design, UML sequence diagrams, and UI wireframes. The fourth work package is Development, which includes the implementation of the Monitoring Agent, the Analysis and Diagnostic Agent, the Data Retrieval Agent, the Dashboard, MySQL integration, the LangChain orchestration layer, and the data pipeline and pre-processing module. The fifth work package is Implementation, which includes the deployment of FastAPI and Docker, and connecting the frontend to the backend. The sixth work package is Testing and Evaluation, which comprises unit testing, integration testing, anomaly detection evaluation, diagnostic quality assessment, and performance and latency testing. The seventh and final work package is Documentation, which includes the project proposal, contextual report, reflective report, and the final thesis report.

## **4 Section 4: Planning Your Artifact**

### **4.1 Methodology**

Developed using an Agile-inspired methodology and two-week long sprints. This was decided for three primary reasons. First, flexibility is the result because enterprise needs may change due to feedback from stakeholders during the development process. Secondly, incremental delivery enables the core functionality (such as the Monitoring Agent) to be delivered early in the project's timeline. Third, risk reduction is possible, since the risk of fail to integrate is minimized towards the end of the project due to regular testing and validation at the end of each sprint. The methodology incorporates aspects of Design Science Research (DSR) used to create and evaluate artifacts, and Rapid Application Development (RAD) used to rapidly develop a prototype. Design Science Research is a well suited method as the artifact, "Fixora", is a new artifact that will help solve a real world problem in the enterprise system administration field, and evaluating the artifact against the intended goals is a central component of the research contribution. Rapid Application Development elements are added to guarantee that a working prototype is delivered in the twelve weeks and that the user feedback and iterative refinement process is followed.

### **4.2 Planning**

The project planning is based on the Gantt chart and Work Breakdown Structure defined in Section 3 of this report. Some of the main planning decisions involve resources allocation, tracking of milestones, selection of tools and protocols for communication. As far as resource allocation goes, the project allocates about 15 weeks for development, which is split into 3 sprints. With regard to milestone tracking, weekly progress reviews are held with the researcher's supervisor to keep the project on track and to deal with any issues that arise quickly. As for the tools, JIRA is utilized for tasks tracking and sprinting, whereas GitHub is used for all artefacts of code version control. As far as communication is concerned, monthly progress reports are handed to the supervisor, while bi-weeksly meetings are set up to discuss technical problems in detail and make design decisions. The planning also provides for contingencies of about fifteen percent of the total effort in case of unforeseen delays or technical problems.

### **4.3 Requirement Gathering and Analysis**

The needs that surfaced for the Fixora project were collected in three main ways. The initial channel included an 84 industry expert survey, gathering quantitative and qualitative information on user pain points, feature desires, and system constraints. The second channel was stakeholder interviews, which comprised of follow-up interviews with five of the survey respondents who indicated they were willing to give more detailed feedback on their experiences with enterprise system administration. The third channel involved competitive analysis, examining the tools available in the market already (Datadog, New Relic, Splunk) to see where there were gaps to be filled by Fixora. The following are the key functional requirements, identified as a result of this process. Based on the survey results of ninety-two percent of respondents, who approved this feature, it was assigned high priority and shall be detected automatically in real-time. The system shall allow for natural language explanations of errors where the error is given high priority based on qualitative feedback that the error requires "plain English" communication. The system shall be able to query databases using natural language, which is of medium priority because it is for non-technical users. Diagnostic reports shall be produced by the system, with medium priority for the use of the IT administrator. The system will be integrated with MySQL databases and will be assigned low priority as this is for prototype environment. The following key non-functional requirements have been identified. Less than 5 seconds response time for diagnostic mode, with 95 percent confidence in root cause diagnosed. For steady-state resource usage, the goal is to keep monitoring's CPU usage below two percent of the HOST's CPU usage to provide a "zero impact" monitoring profile. The desired accuracy is to achieve over 90% correct diagnoses in relation to ground truth from experts to ensure diagnostic accuracy. The target for system availability is 99.9 per cent uptime for the core monitoring functions.

### **4.4 Design**

The architecture of Fixora is well designed and follows the principle of decoupled architecture, to ensure the best possible scalability, maintainability and fault tolerance in the modular community. In an enterprise environment where software components

evolve and are replaced, a monolithic design is a big problem. Fixora has a layered architecture separating data ingestion, intelligent processing and user interaction. The Data Ingestion Layer acts as the “eyes and ears” of the system, securely and continually integrated with all kinds of corporate data sources. This layer can be directly plugged into relational databases like MySQL and PostgreSQL, it can follow system logs and monitor the API's "heartbeat". This layer is used to combine ingestion data from multiple sources, including transaction data, server logs and more, giving the following AI layers a high-resolution, real-time view of the entire digital ecosystem. This layer is connected to the Orchestration Layer, which is the "nervous system" of the artifact. Developed using the LangChain toolkit, this layer is responsible for intent matching and agent selection. The Orchestration Layer receives a natural language question from a user and determines the linguistic intent of the user and identifies which agent(s) have the knowledge needed to complete the question. An important part of this separation is its ability to add new agents to the system without requiring any changes to the logic on the backend. Finally, the Agent Layer is the intelligence units specialization that perform administrative work. In this, each agent operates individually in its specialty and shares the context with the other members of the team in a coordinated way with the orchestrator. The quality, clarity, and precision of Fixora's agents and the interactions between them is what makes it strong. Unlike traditional applications with static functions, agents rely on Large Language Models (LLMs) to make decisions in a dynamic and context-aware manner. The Monitoring Agent is the watchful eye in the front line, and not only alerts when CPU usage reaches ninety percent for more than three minutes, it also employs advanced anomaly detection based on AI. This agent can see that durations of the database connections are beginning to slowly rise and even by historical patterns, this could mean that it will cause the entire system to fail. It is primarily used to “raise the alarm” with contextual information before it is a big problem. If a variability is detected, the Diagnostic Agent is started. This agent is the one that does the "forensic" work that consumes a lot of an IT specialist's time at present. Scans the relevant logs marked by the Monitoring Agent and performs root cause analysis with the help of a Large Language Model. Does not return an error code, but rather a human-readable message. In the above scenario, the Diagnostic Agent would not raise a “SocketException”, but rather, the message would be “The payment

gateway is down because the SSL certificate expired at midnight”. The Fixora system consists of this change from information to intelligence. The third agent is the Retrieval Agent who assists people who are not as technologically advanced. This is a natural language interface to the structured data of the business so that a business user can ask a question like "How many orders are pending?". The agent transforms this into a SQL query, queries a MySQL database and returns the result in natural language. This eliminates the need to learn the tricky syntax of the query or to rely on an IT person to execute a manual report.

#### **4.5 Implementation**

Fixora's tech stack is selected to ensure that it's up to date in terms of artificial intelligence technology and dependable enough for enterprise-scale software. The Backend is built using Python, a most popular programming language for AI and data science, along with FastAPI. FastAPI was selected due to its speed and built-in asynchronous programming support, which is important when managing several AI agents that may take different amounts of time to respond. To facilitate AI Orchestration, the project uses LangChain, offering the requisite functionality for "agent memory" and "chain-of-thought" processing. Grok and GPT-4o are powerful models that perform the natural language processing via secure API calls. These models have the ability to interpret and interpret intricate user input and generate technically valid diagnostic reports. The Frontend is a responsive admin dashboard built with React.js that displays the status of the system, alert count and has a chat interface to communicate with the agents. It is influenced by a "Cottagecore" design choice, focusing on simplicity, comfort and user-friendliness, to minimize the stress that is often associated with technical problems. Lastly, there are two solutions to the database strategy: SQLite is used for the prototype's internal agent memory, and for local testing, while a full-fledged MySQL database is used to simulate the enterprise environment. This ensures that the artifact is tested with the same relational structure that will be employed in the enterprise and is ready to be deployed in the enterprise.

The implementation plan of the Fixora project have three Agile sprints as per the Gantt chart, running from April to August 2026. Sprint 01 (20 April – 15 May 2026) is

dedicated to Architecture and UI, covering project plan preparation and artefact planning, establishing the system blueprint and design foundations. Sprint 02 (2 June – 26 June 2026) focuses on the Intelligence Layer, with the development of the Monitoring Agent for log ingestion and the Analysis Agent for anomaly detection. Sprint 03 (29 June – 7 August 2026) addresses Interaction and Prototype delivery, encompassing the LangChain orchestration layer, the natural language Data Retrieval Agent, the System Health Dashboard integration, and prototype finalization. Following the sprints, Testing and Evaluation (10 August – 21 September 2026) covers unit and integration testing, User Acceptance Testing, and performance testing. Final Report and Documentation runs concurrently throughout the entire project from February to September 2026, with progress reports, the contextual report, the reflective report, and the final thesis report produced at key milestones.

#### **4.6 Testing and Evaluation Strategy**

In evaluating an automated administrative system like Fixora, one should take two steps to catch the elements of performance evaluation in high-precision-control systems. In control systems engineering, performance is a measure of the transient response and steady-state response; for a multi-agent AI system to manage enterprise activities, it's the response of the system to unexpected technical emergencies and the managerial oversight of the system under normal operating conditions. The critical period after the first occurrence of a fault in the system is called the “transient” phase. This is typically the stage when chaos and delays occur in a conventional enterprise system, as system administrators gather information to diagnose the fault. The Fixora artifact seeks to change this time. When a fault occurs in the enterprise computing environment, such as when the number of lock waits in the database rises suddenly or when a large number of HTTP errors are returned at 500-level errors, then the system should switch from background monitoring mode to “Diagnostic Mode.” For this transition the key performance indicator is "Time to Confidence". The aim of the Fixora Diagnostic Agent is to have a 95% certainty of root cause in 5 seconds. This includes the agent's ability to rapidly process the log events that have been asynchronous and the real-time measurement of the systems and known error trends of the past. This way, Fixora cuts the "fog of war" that can problems the first few minutes of the IT incident, allowing an



almost instant right-turn to problem resolution. Normal and healthy behavior, however, is called "steady-state. The monitoring system should not affect the health of the system being monitored. If the monitoring agent uses significant amounts of CPU or memory resources, it can create the issue it's looking to resolve. As a result, the steady-state objective of Fixora's Monitoring Agent is to have a "Zero-Impact" profile. One of the ways to do this is to make sure the agent doesn't use more than two percent of the total computing power of the host system in steady state. This is achieved with the help of low weight telemetry collectors and data-streaming protocols. This means that the AI is not intrusive, and only shows itself when it is required, providing round-the-clock monitoring without interfering with the business-critical applications that the AI monitors, including enterprise resource planning or customer relationship management systems. There are four distinct testing activities outlined in the testing strategy. The first one is the transient state testing, which means the simulation of faults is injected to the test environment and the response time of the diagnostic is measured. Second, the steady-state test which consists of running the monitoring agent for 72 hours and measuring its CPU and memory utilization to ensure that it is below 2 percent. Third, diagnostic accuracy testing requires comparing Fixora's diagnosis with the expert-checked fault set of fifty test cases to obtain more than 90% accuracy. Fourth, user acceptance testing (UAT) is the ten non-technical users executing the normal tasks and providing the System Usability Scale scores, targeted to be greater than seventy.

#### 4.6 Risks and Risk Mitigation

When implementing a multi-agent AI system in a real enterprise setting, there are a number of risks to be identified and mitigated with built-in features. A risk assessment has been conducted as part of the assessment of Fixora, which is summarized in Table

**Table 2: Risk Assessment and Mitigation Strategies**

| Risk Category | Risk Description  | Probability | Impact | Mitigation Strategy        |
|---------------|-------------------|-------------|--------|----------------------------|
| Technical     | LLM Hallucination | Medium      | High   | Use RAG to constrain AI to |

| Risk Category | Risk Description                                        | Probability | Impact | Mitigation Strategy                                               |
|---------------|---------------------------------------------------------|-------------|--------|-------------------------------------------------------------------|
|               | producing incorrect diagnoses                           |             |        | factual data; require evidence citation                           |
| Security      | Unauthorized data access via natural language interface | Low         | High   | Implement Role-Based Access Control (RBAC) at Orchestration Layer |
| Performance   | Agent overhead impacting system performance             | Medium      | Medium | Optimize monitoring agent to use lightweight telemetry collectors |
| Integration   | Compatibility issues with legacy enterprise systems     | Medium      | Medium | Use standard APIs and database connectors; modular design         |
| Resource      | Insufficient time for full implementation               | Medium      | High   | Prioritize core features (Monitoring + Diagnostic agents)         |
| User Adoption | Non-technical users reluctant to trust AI               | Low         | Medium | Provide transparent explanations; gradual feature rollout         |

One of the biggest challenges in an administrative system using AI is that data can be accessed by people without authorisation through the natural language interface. Fixora allows users to ask natural language questions and retrieve information from the system and from the database; it is possible that an user can be granted access to data that is not within his/her clearance. For instance, a non-supervisor may try to ask questions about payroll figures while checking the health of the system. The main defence in this case is to implement Role-Based Access Control (RBAC) at the Orchestration Layer.

Fixora will verify identity and permissions of the user before it sends a query to the Retrieval Agent. If the query needs to access a restricted table, or higher-level system logs, it will be denied access to the data or part of the data. This guarantees that although system administration is "democratized", it is not subverted. Another potential danger associated with the use of Large Language Models for technical diagnostics is the danger of "hallucination" where the model gives an answer which is technically correct, and "convincing" but really technically impossible or simply wrong. In the realm of IT, a hallucinated diagnosis could prompt an IT admin to reboot the incorrect server or remove a valid database index, exacerbating the problem. It can be addressed through the technology of Retrieval-Augmented Generation (RAG) for diagnostic reasoning, as implemented in Fixora. The Diagnostic Agent is not allowed to use the LLM's built-in knowledge to "infer" the cause of system failure, but is only limited to the information contained within the log and the telemetry data ingested from the Data Ingestion Layer. By requiring the AI to cite evidence for its responses, in this case the particular log line that led to the response, hallucination is prevented. This ensures that the system will give reliable advice, based on the condition of the digital ecosystem of the enterprise. There are also other risks such as performance overhead, integration problems, resource issues, and user adoption concerns with their respective mitigation strategies listed in Table 2. The goal of the Fixora project is to provide a dependable, secure, and effective multi-agent AI system in enterprise administration, by proactively identifying and mitigating risks.

## 5 References

- [1] M. Wooldridge and N. R. Jennings, "Intelligent agents: theory and practice," *The Knowledge Engineering Review*, vol. 10, no. 2, pp. 115-152, 1995.
- [2] P. Dang, F. Lin, and J. Huang, "AIOps: Real-world Challenges and Solutions," in *Proc. IEEE International Conference on Software Engineering*, 2019.
- [3] R. M. I. T. Rathnayake, "Fixora - Multi-Agent Artificial Intelligence-Based Automated Administrative System for Enterprise Business Operations," BSc (Hons) Computer Science, University of Bedfordshire, 2026.
- [4] N. R. Jennings, K. Sycara, and M. Wooldridge, "A Roadmap of Agent Research and Development," *Autonomous Agents and Multi-Agent Systems*, vol. 1, no. 1, pp. 7-38, 1998.
- [5] Dr. Logeeshan V., "Design PID controller for a LAMP tester heating module by using PLC/FPGA," Kingston University, School of Engineering, 2026.
- [6] R. M. I. T. Rathnayake, "Enterprise System Challenges & AI-Based Support (Responses)," primary research survey data, April 2026.

## 6 Appendices

### 6.1 Appendix A: Questionnaire/Interview Questions

<https://docs.google.com/forms/d/e/1FAIpQLSck-eSIdLXOr5ky7oZUSwl75R9UHZBSqE2D8EsUasVn5NpZ6g/viewform?usp=sharing&ouid=101173257333573254385>



## Enterprise System Challenges & AI-Based Support

This survey aims to understand challenges faced by both business users and IT professionals when using and managing enterprise systems (e.g., ERP, SAP). It also evaluates the need for AI-based solutions for issue detection, troubleshooting, and automation.

All responses will remain confidential and will be used only for academic research.

[Sign in to Google](#) to save your progress. [Learn more](#)

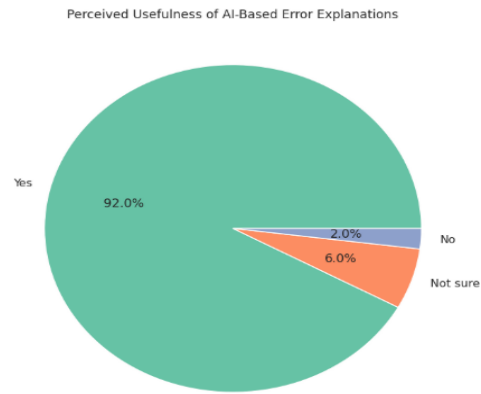
\* Indicates required question

**What is your role? \***

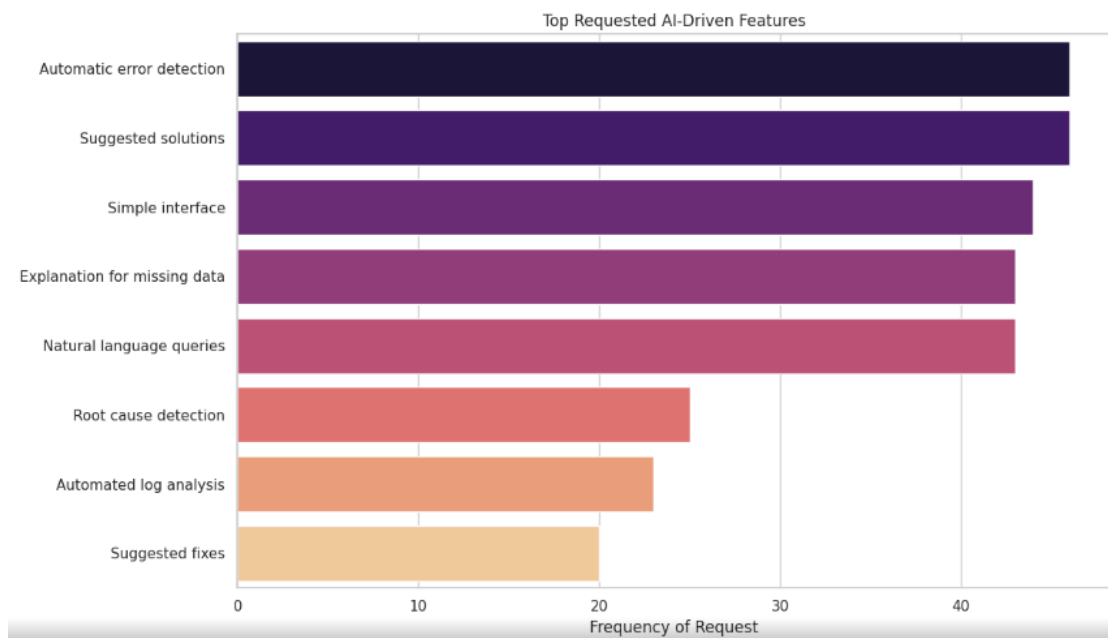
☐ Non-IT Staff (Business/User)

☒ IT staff / Developers

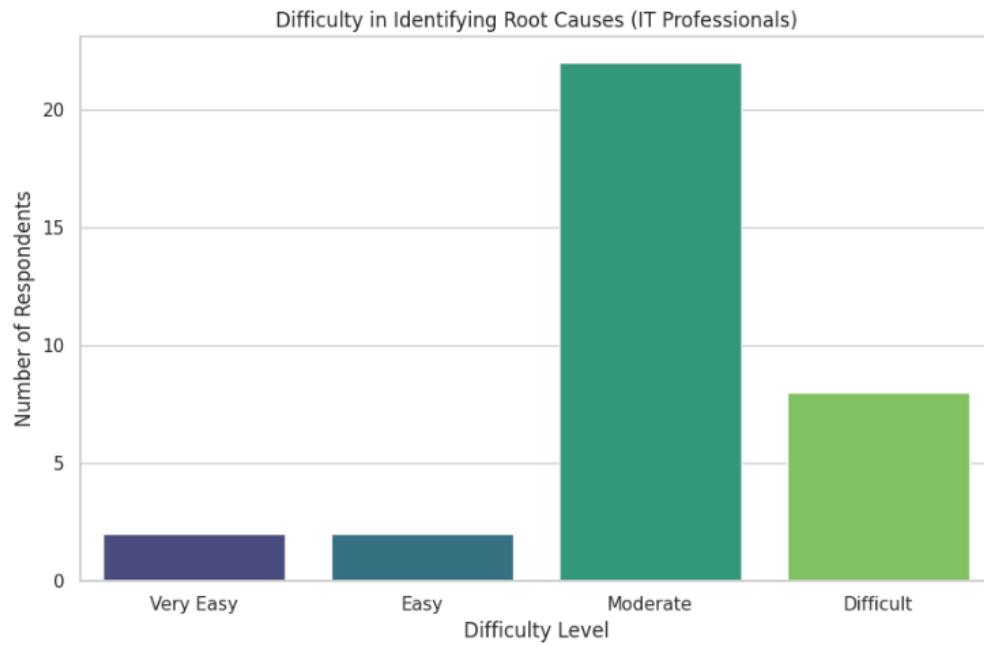
*Figure 3: Questions include google sheet for survive*



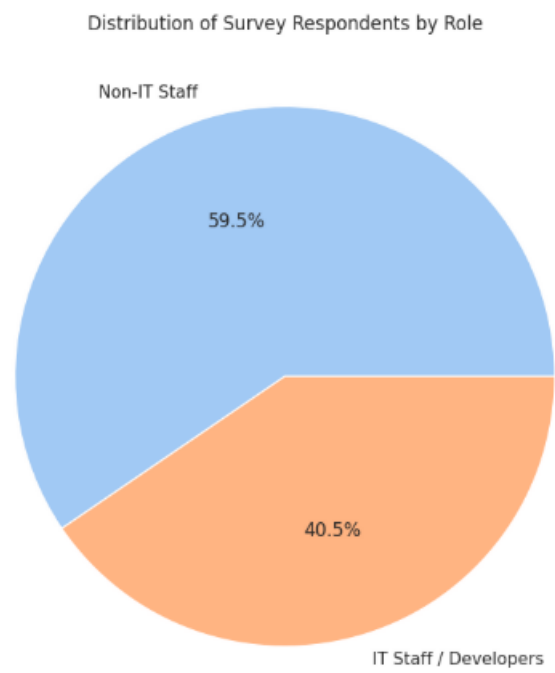
*Figure 4 : Role Distribution of Responds*



*Figure 5 : Top Requested AI-Driven Features*



*Figure 6 : Weekly Time Lost due to System Inefficiencies*



*Figure 7 : Perceived Usefulness of AI-Based Error Explanations*



*Figure 8 : SWOT analysis*